Food Calorie & Nutrient Estimator — Full Pipeline Guide

Course: Deep Learning — Semester 02
Team: Emiel & Luca (CV Pipeline) | Mahesh & Furaha (VLM Refinement)
Date: April 2026

---

1. Project Overview

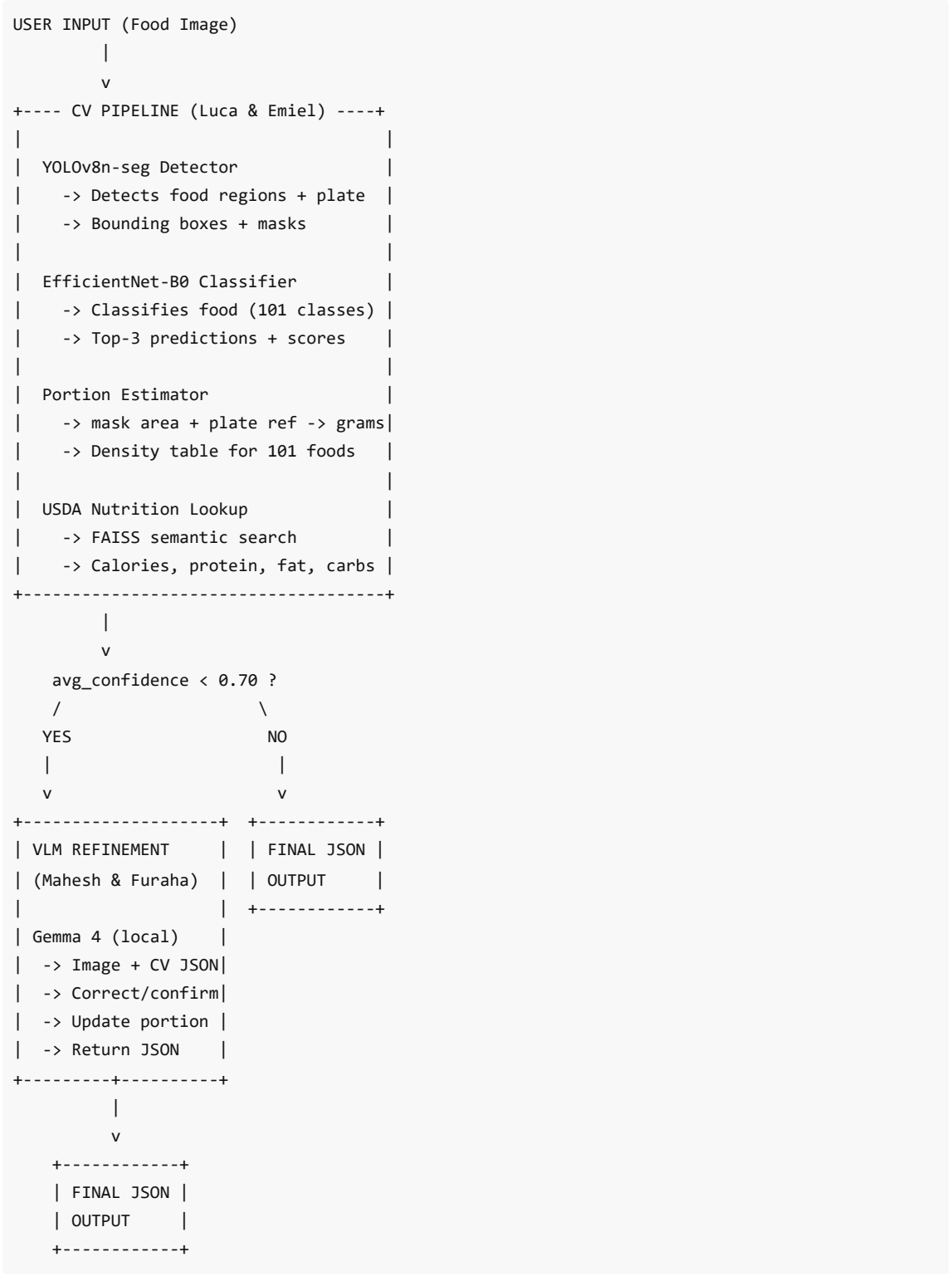
This project builds an **AI-powered food detection and nutrition estimation system** using a hybrid CV + VLM architecture. The system takes a photo of a meal, detects food items, estimates portion sizes, and calculates nutritional information (calories, protein, fat, carbs, fiber).

Why Hybrid?

Traditional CV pipelines (YOLO + classifiers) are fast but struggle with ambiguous foods. Vision Language Models understand context better but are slower. Our system uses the CV pipeline first and calls the VLM only when confidence is low — getting the best of both worlds.

---

2. Architecture



---

3. CV Pipeline — Detailed

3.1 YOLOv8n-seg Detector (Phase 3 — Not Implemented)

File: `detector/__init__.py` (empty)

Purpose: Detect food items and plates in the image using instance segmentation.

Input: Raw image
Output per detection:

- Bounding box: `[x1, y1, x2, y2]`
- Segmentation mask: binary numpy array
- Detection confidence: float 0-1
- Class: "food" or "plate"

Technology: Ultralytics YOLOv8n-seg (smallest variant for speed)

3.2 EfficientNet-B0 Classifier (Phase 2 — Complete)

File: `classifier/classifier.py`
Accuracy: 87.37% on Food-101 test set

Architecture:

- Base: EfficientNet-B0 (ImageNet architecture, random weights)
- Custom head: Dropout(0.2) -> Linear(1280,512) -> SiLU -> Dropout(0.2) -> Linear(512,101)
- Output: 101 food classes

Code example:

```
from classifier.classifier import FoodClassifier

classifier = FoodClassifier(
    model_path="models/efficientnet_b0_food101_best.pt",
    labels_path="models/classifier/idx_to_class.json"
)

results = classifier.predict(cropped_image, top_k=3)
# [{"label": "fried_rice", "confidence": 0.58}, ...]
```

Preprocessing: Resize(256) -> CenterCrop(224) -> ToTensor -> Normalize(ImageNet mean/std)

3.3 Portion Estimator (Phase 4A — Complete)

File: `portion/portion.py`

Method: Geometry-based estimation using segmentation masks.

Calculation steps:

1. Get food mask pixel area from YOLO segmentation
2. If plate detected: calculate cm_per_px from plate diameter (26cm default)
3. If no plate: use fallback scale (assume food is 40% of plate area)
4. Calculate: `food_area_cm2 = food_pixels * (cm_per_px)^2`
5. Look up food-specific height and density from density table
6. Calculate: `grams = food_area_cm2 * height_cm * density_g/cm3`
7. Clamp to range: 10g - 1500g

Density table: All 101 Food-101 classes with height_cm and density_g/cm3.

```python
from portion.portion import PortionEstimator

estimator = PortionEstimator()
result = estimator.estimate("fried_rice", food_mask, plate_mask)
# {"estimated_grams": 185.0, "portion_method": "plate_reference"}
```

3.4 USDA Nutrition Lookup (Phase 4B — Complete)

File: `nutrition/nutrition.py`

Method: Semantic search using FAISS + sentence-transformers.

1. Food name -> sentence-transformer embedding (all-MiniLM-L6-v2)
2. FAISS index search against USDA FoodData Central
3. Returns closest match with nutrition per 100g

```python
from nutrition.nutrition import NutritionLookup

lookup = NutritionLookup(
    index_path="data/usda/faiss_index.bin",
    records_path="data/usda/records.json"
)

result = lookup.lookup("fried_rice")
# {"usda_description": "Fried rice", "nutrition_per_100g": {...}}
```

---

4. VLM Refinement Stage — Detailed

4.1 Module Structure

```
vlm/
  __init__.py          - Module header
  schemas.py           - Pydantic models (input/output)
  prompts.py           - System + user prompt templates
  client.py            - Gemma 4 API client
  refiner.py           - Main orchestrator
  example_input.json   - Sample CV output
  example_output.json  - Sample VLM response
```

4.2 Model: Gemma 4 E4B (Open-Source)

Property	Value
Model	Gemma 4 E4B (effective 4B parameters)
Family	Google DeepMind open models
Quantization	Q4_K_M (~3 GB disk, ~11 GB RAM)

Input	Text + Image (multimodal)
Context	128K tokens
Backend	Ollama (local, free, offline)
Inference	~2-4 min per image (CPU+GPU hybrid)
Cost	$0.00

Why Gemma 4 E4B?

- Fits on laptop (32GB RAM, T500 4GB GPU)
- Strong vision capabilities
- Fully offline, no API key needed
- Ollama handles GPU/CPU split automatically

4.3 JSON Schemas (schemas.py)

All data is validated using **Pydantic v2** models for type safety.

Input (VLMRequest) — sent from CV pipeline:

```
{
  "image_base64": "base64-encoded image...",
  "reason": "low_confidence",
  "trigger_threshold": 0.70,
  "cv_output": {
    "image_id": "uuid",
    "average_confidence": 0.55,
    "plate_detected": true,
    "items": [{
      "item_id": 1,
      "food_name": "spaghetti_bolognese",
      "classification_confidence": 0.55,
      "estimated_grams": 250,
      "nutrition_per_100g": {
        "calories_kcal": 132,
        "protein_g": 5.8,
        "fat_g": 4.5,
        "carbs_g": 18.1,
        "fiber_g": 1.4
      }
    }]
  }
}
```

Output (VLMResponse) — returned by VLM:

```
{
  "image_id": "uuid",
  "refinement_status": "completed",
  "vlm_model": "gemma4:e4b",
  "items": [{
    "item_id": 1,
    "action": "corrected",
    "original": {"food_name": "spaghetti_bolognese",
                 "classification_confidence": 0.55},
    "refined": {"food_name": "pancakes",
                "display_name": "Pancakes",
                "vlm_confidence": 0.92},
    "portion": {"estimated_grams": 250,
                "portion_method": "vlm_visual_estimate",
                "vlm_confidence": 0.85},
    "nutrition_per_100g": {...},
    "nutrition_total": {...}
  }],
  "totals": {...},
```

```
  "processing_time_ms": 214000
}
```

Key Enums:

- **RefinementAction**: confirmed | corrected | unknown
- **RefinementReason**: low_confidence | multiple_similar | no_food_detected | portion_suspect
- **PortionMethod**: plate_reference | fallback_scale | vlm_visual_estimate

---

4.4 Prompts (prompts.py)

System Prompt tells Gemma 4:

1. It's a food identification expert
2. Must use exact Food-101 class names (all 101 listed)
3. Must respond with JSON only
4. Must set honest confidence values (0.3-1.0, not always 0.95)

User Prompt tells Gemma 4:

1. What the CV pipeline predicted (e.g., "spaghetti_bolognese at 55%")
2. To confirm or correct the prediction
3. To respond with structured JSON

Two prompt versions:

- **FAST prompts** — shorter, faster (2-4 min), used for evaluation
- **FULL prompts** — detailed with nutrition, slower (5-10 min), used for production

---

4.5 Client (client.py)

`Gemma4Client` wraps the Ollama API for Gemma 4 communication.

Supported backends:

Backend	Config	Cost	Speed
Ollama (local)	backend="ollama"	Free	2-4 min
Google AI Studio	backend="google" + API key	Free tier	~10 sec

API call flow:

```
client.py
  -> HTTP POST localhost:11434/api/chat
  -> {model, messages, stream:false, options:{temp, top_p, top_k}}
  -> Ollama loads model into GPU/CPU memory
  -> Gemma 4 processes image + text
  -> Returns JSON response
```

JSON extraction: Handles direct JSON, markdown code blocks (```json), and embedded JSON.

4.6 Refiner (refiner.py)

`VLMRefiner` orchestrates the full refinement flow:

```
Step 1: Validate input   -> VLMRequest (Pydantic validates all fields)
Step 2: Build prompt     -> System + user prompt with CV data
Step 3: Call Gemma 4     -> Via client.py (Ollama or Google)
Step 4: Parse response   -> Extract items, map to CV items by item_id
Step 5: Calculate totals -> Sum nutrition across all refined items
Step 6: Return           -> VLMResponse (validated by Pydantic)
```

```python
from vlm.refiner import VLMRefiner

refiner = VLMRefiner(backend="ollama", model="gemma4:e4b")
result = refiner.refine(request_dict)
print(result.model_dump_json(indent=2))
```

---

5. Live Test Results

Detail	Value
Model	gemma4:e4b (Ollama, fully local)
Image	Toast/pancake dish
CV Prediction	spaghetti_bolognese (55%) — WRONG
VLM Correction	Cheese Pancakes (95%) — CORRECT
Action	corrected
Portion	250g, 280 kcal/100g
Time	214 seconds
Cost	$0.00

---

6. Evaluation Setup

10 food images with deliberate mix of correct and wrong CV predictions:

Image	Actual Food	CV Predicts	Correct?
pizza.jpg	pizza	pizza	Yes (confirm)
burger.jpg	hamburger	hot_dog	No (correct)
sushi.jpg	sushi	sushi	Yes (confirm)
pasta.jpg	spaghetti_bolognese	ramen	No (correct)
salad.jpg	greek_salad	caesar_salad	No (correct)
pancakes.jpg	pancakes	french_toast	No (correct)

icecream.jpg	ice_cream	frozen_yogurt	No (correct)
steak.jpg	steak	filet_mignon	Close
ramen.jpg	ramen	pho	No (correct)
cake.jpg	chocolate_cake	red_velvet_cake	No (correct)

Run: `python -u scripts/eval_vlm.py`

---

7. Quick Reference — All Commands

```
# Setup
venv\Scripts\activate

# Offline tests (instant)
python scripts/test_vlm.py
python -u playground.py

# Live VLM test (2-4 min)
python -u scripts/test_vlm_fast.py
python -u scripts/test_vlm_fast.py path\to\photo.jpg

# Full evaluation (~30 min)
python -u scripts/eval_vlm.py

# Luca's tests
python scripts/test_nutrition.py
python scripts/test_portion.py
```

---

8. Project Status

Phase	Component	Status	Author
1	Environment Setup	Done	Luca
2	EfficientNet-B0 Classifier	Done (87.37%)	Luca
3	YOLOv8n-seg Detector	Not started	Luca
4A	Portion Estimator	Done	Luca
4B	USDA Nutrition Lookup	Done	Luca
5	FastAPI Integration	Not started	Luca
6	VLM Refinement Module	Done	Mahesh & Furaha
7	Live Testing	Done	Mahesh
8	Multi-image Evaluation	Ready to run	Mahesh & Furaha

---

9. Team Contributions

Luca (6 commits, pushed to GitHub)

- Project structure, dependencies, git setup
- EfficientNet-B0 classifier trained on Food-101 (87.37%)
- Portion estimator with density table for all 101 classes
- USDA FAISS nutrition lookup with sentence-transformers
- README, integration tests

Mahesh & Furaha (local, pending push)

- VLM module: schemas, prompts, client, refiner (7 files)
- JSON interface design (Pydantic input/output schemas)
- Gemma 4 integration via Ollama (fully open-source)
- Test scripts: offline, local, fast, online (4 scripts)
- Evaluation framework: 10 food images + automated scoring
- Live test: successful correction of wrong CV predictions
- Playground walkthrough for data flow understanding

---

10. Key Technical Decisions

Decision	Choice	Reason
VLM Model	Gemma 4 E4B	Open-source, fits on laptop, good vision
Backend	Ollama (local)	Free, no API key, offline capable
Validation	Pydantic v2	Type safety, auto-validation
Prompt Strategy	FAST + Food-101 list	5x faster, exact class names
Confidence Threshold	0.70	To be agreed with Luca
JSON Parsing	Multi-strategy	Handles markdown, embedded, raw JSON